

PROJECT REPORT

on

TRAFFIC VIOLATION REPORTING SYSTEM

A MERN Stack Web Application

(MongoDB, Express.js, React.js, Node.js)

Submitted by

K. Akshaya—25MVCSMR0048

L. Akshaya—25MVCSMR0051

M. Nandini—25MVCSMR0057

P. Gangothri—25MVCSMR0069

Department of CSE (AI/ML)

Mallareddy Technical
campus

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide for the continuous guidance, encouragement and constructive feedback provided throughout the development of this project. The suggestions received during the review sessions were instrumental in shaping the design of the system and in improving the quality of this report.

I am also thankful to the Head of the Department, the faculty members and the laboratory staff for providing the infrastructure and the technical environment that were needed to complete the work. I extend my thanks to my friends and classmates who took part in usability testing and offered valuable observations about the interface and workflow.

Finally, I thank my family for their patience and support. Any shortcomings that remain in this work are entirely my own.

TABLE OF CONTENTS

Chapter 1: Abstract and Introduction

Chapter 2: Problem Statement, Objectives and Scope

Chapter 3: Features and Functional Requirements

Chapter 4: Algorithms and Methodology

Chapter 5: Technologies and Implementation

Chapter 6: Results, Testing and Performance

Chapter 7: Conclusion, Future Scope and References

Each chapter is divided into numbered sections. The chapters follow the order of the project life cycle: understanding the problem, specifying the requirements, designing the solution, implementing it, and finally evaluating it.

CHAPTER 1: ABSTRACT AND INTRODUCTION

1. Abstract

Road traffic violations such as signal jumping, riding without a helmet, over-speeding, wrong-side driving and illegal parking are among the leading causes of road accidents and congestion in urban areas. Enforcement is largely manual and depends on the physical presence of traffic personnel, which is limited by manpower and by geography. At the same time, many citizens witness violations every day but have no convenient channel through which they can report them with enough evidence for authorities to act.

This project presents the design and implementation of a Traffic Violation Reporting System built on the MERN stack. The application allows registered citizens to report a violation by providing the vehicle registration number, the type of violation, the location, the date and time, and a photographic evidence. The report is stored in MongoDB and routed to traffic officers, who verify the evidence, accept or reject the report, and issue a challan with a fine amount. Administrators manage users, violation categories and fine schedules, and view statistical dashboards that summarise violation trends by type, area and time.

The backend is a RESTful service built with Node.js and Express.js. It uses JSON Web Tokens for stateless authentication and role-based access control to separate the privileges of citizens, officers and administrators. Images are accepted through multipart uploads, validated for type and size, and stored with their metadata. The frontend is a single-page application built with React.js, offering responsive forms, searchable and filterable lists, status tracking and charts. The system supports complete CRUD operations on vehicles, violations, locations and evidence records. Functional testing, API testing and performance testing show that the system behaves correctly under the specified scenarios and responds within acceptable limits for typical loads.

Keywords: Traffic violation, MERN stack, REST API, JWT authentication, role-based access control, MongoDB, image upload, e-challan.

2. Introduction

Rapid urbanisation and growth in vehicle ownership have placed enormous pressure on road networks. India alone registers several million new vehicles every year, and traffic density in metropolitan cities frequently exceeds the capacity for which roads were designed. Under such conditions, disciplined driving is the single most important factor in maintaining road safety. Traffic rules exist to protect road users, yet violations remain common, partly because the probability of being caught is low.

Traditional enforcement relies on personnel stationed at junctions and on fixed cameras at a limited number of locations. Both approaches have coverage gaps. A violation that occurs on a side street or outside peak hours will usually go unrecorded. Citizens, on the other hand, are everywhere, and almost every citizen carries a smartphone capable of capturing photographs with a time stamp. A well-designed reporting platform can convert this latent capacity into a structured stream of evidence that supplements official enforcement.

Web technologies make such a platform feasible at low cost. The MERN stack, a combination of MongoDB, Express.js, React.js and Node.js, allows a complete application to be built with a single language, JavaScript, across the database layer, the server and the client. This uniformity shortens the learning curve, enables code and data-format reuse, and makes it easier to maintain the system. The document-oriented nature of MongoDB suits this domain well, because a violation report is naturally a self-contained document that contains nested information about the vehicle, the location and the evidence.

3. Background and Motivation

Several cities have launched mobile applications and helplines through which citizens can send photographs of violations. Experience from these initiatives highlights recurring difficulties. Reports arrive in unstructured form through messaging applications, so officers must manually extract the vehicle number, location and time. There is no consistent record of what happened to each complaint, which discourages citizens from reporting again. Duplicate reports of the same incident are common and waste verification effort. Finally, there is little analytical insight into which locations and violation types are most problematic.

The motivation for this project is to address these difficulties through a purpose-built platform in which every report is captured in a structured schema, passes through a well-defined life cycle, and is available for analysis. The project is also motivated by the academic goal of demonstrating a complete full-stack workflow, from requirement analysis and data modelling to API design, user-interface development, security and testing.

4. Overview of the Proposed System

The proposed system has three classes of users. A Citizen registers, logs in, submits violation reports with evidence, and tracks the status of each report. A Traffic Officer reviews pending reports, examines the evidence, verifies the vehicle details, and either approves the report with a fine or rejects it with a reason. An Administrator manages user accounts, configures violation types and fine amounts, and reviews system-wide statistics.

A report moves through the states Submitted, Under Review, Approved, Rejected and Closed. Each transition is recorded with the identity of the actor and a time stamp, which creates an audit trail. The system exposes more than twenty REST endpoints, all of which validate input, enforce authorisation and return consistent JSON responses.

Aspect	Description
Project title	Traffic Violation Reporting System
Architecture	Three-tier, client-server, REST based single-page application
Frontend	React.js with React Router, Axios and Chart.js
Backend	Node.js with Express.js, Mongoose ODM
Database	MongoDB (collections: users, vehicles, violations, violationtypes, locations)
Security	bcrypt password hashing, JWT, role-based access control, input validation
Users	Citizen, Traffic Officer, Administrator
Core entities	Vehicle, Violation, Location, Date/Time, Image evidence

Table 1.1: Summary of the proposed system

1.5 Organisation of the Report

Chapter 2 states the problem, objectives and scope. Chapter 3 lists the features together with functional and non-functional requirements. Chapter 4 explains the methodology, the architecture, the database design and the key algorithms. Chapter 5 describes the technologies and the implementation with representative source code. Chapter 6 presents the testing strategy, results and performance observations. Chapter 7 concludes the report, outlines the future scope and lists the references.

CHAPTER 2: PROBLEM STATEMENT, OBJECTIVES AND SCOPE

1. Problem Statement

To design and develop a MERN-based web application for reporting traffic violations, in which users can submit and manage reports consisting of the vehicle, violation type, location, date and image evidence, and in which authorised officers can review, update and manage these records through secure, role-based interfaces and REST APIs backed by MongoDB.

1. Problems with the existing approach

- Limited coverage: officers and cameras cannot be present at every location and time.
- Unstructured complaints: information arrives through calls or messaging apps and has to be re-entered manually.
- No tracking: a citizen who reports a violation has no way to learn whether any action was taken.
- Evidence handling: photographs are scattered across devices and are difficult to associate with a case.
- Duplicate effort: the same incident can be reported by several witnesses and verified several times.
- Lack of analytics: absence of searchable records makes it hard to identify accident-prone locations or repeat offenders.
- Accountability: manual processes leave no audit trail of who approved or rejected a complaint.

2. Proposed solution

The proposed solution is a centralised web platform that standardises the capture of violation reports, preserves evidence in a structured form, automates routing and status updates, and provides dashboards for decision making. By making reporting easy for the public and verification efficient for officers, the system aims to increase the perceived likelihood of enforcement and thereby encourage compliance.

2. Objectives

The principal objective is to demonstrate the development of a complete MERN application that solves a real civic problem. The specific objectives are listed below.

- **Demonstrate MERN stack development:** integrate MongoDB, Express.js, React.js and Node.js into a coherent application with a clear separation of concerns.
- **Implement CRUD operations:** provide create, read, update and delete operations for vehicles, violations, locations and users using Mongoose models.
- **Build REST APIs:** design resource-oriented endpoints with proper HTTP verbs, status codes, validation and error handling.
- **Develop the frontend:** create a responsive React interface with forms, lists, filters, status badges and dashboards.
- **Implement authentication and authorisation:** secure the system with JWT and role-based access for citizens, officers and administrators.
- **Handle image evidence:** accept, validate, store and display photographic evidence associated with each violation.
- **Display information and results clearly:** present reports, statuses, statistics and charts in an understandable manner.

- **Ensure quality:** verify the system through unit, API and functional tests and measure its performance.

3. Scope of the Project

1. In scope

- User registration, login, logout and profile management.
- Role-based dashboards for citizens, officers and administrators.
- Submission of violation reports with vehicle number, violation type, location, date and time, description and image.
- Verification workflow with approval, rejection and fine assignment.
- Management of violation types, fine schedules and locations.
- Search, filter, sort and pagination of reports.
- Statistics: counts by type, status, location and month.
- Duplicate detection based on vehicle, type, location and time proximity.
- Audit history of status changes.

2. Out of scope

- Automatic number-plate recognition and automatic violation detection using computer vision.
- Integration with the official vehicle registration database and payment gateways. These are represented by simulated interfaces.
- Native mobile applications. The web interface is responsive and can be used from mobile browsers.
- Legal adjudication, which remains the responsibility of the competent authority.

4. Assumptions and Constraints

- Users have access to a modern browser and an internet connection.
- Photographs are taken by users and may be of varying quality; officers make the final judgement.
- Fine amounts in the system are configurable sample values used for demonstration and do not represent official legal figures.
- The prototype is deployed on a single server with a single MongoDB instance.

2.5 Feasibility Study

Type	Assessment
Technical	All technologies are open source, well documented and supported by large communities. Required skills (JavaScript, REST, NoSQL) are available.
Economic	No licence cost. Development and testing run on a standard laptop. Hosting can use free or low-cost tiers.
Operational	The interface is simple enough for citizens; officers use a familiar list and detail workflow. Little training is needed.
Schedule	The modular design allows incremental delivery within one academic semester.
Legal	Personal data is minimised and protected; the system only records and forwards evidence and does not itself impose penalties.

Table 2.1: Feasibility analysis

CHAPTER 3: FEATURES AND FUNCTIONAL REQUIREMENTS

3.1 Stakeholders and Roles

Role	Responsibilities	Key permissions
Citizen	Reports violations and monitors their status	Create report; view, edit or delete own pending reports; view own history
Traffic Officer	Verifies reports and issues challans	View all reports; change status; assign fine; manage vehicles and locations
Administrator	Manages the platform	All officer permissions plus user management, violation types, fine schedule, analytics

Table 3.1: User roles and privileges

2. Feature Description

1. Authentication and account management

Users create an account with name, email, phone number and password. Passwords are hashed with bcrypt before storage. On login the server issues a signed JSON Web Token containing the user identifier and role. The client stores the token and attaches it to every API request. Sessions expire after a configurable period, after which the user must log in again. Users can view and update their profile and change their password.

2. Violation reporting

The report form collects the vehicle registration number, the violation type chosen from a list, the location (address text, area and optional latitude and longitude), the date and time of the incident, an optional description, and one image as evidence. The form validates the registration number against the standard Indian format, ensures that the date is not in the future, and checks the image type and size before submission. After submission the citizen receives a unique reference number such as TVR-2026-000123.

3. Vehicle management

Each vehicle is stored once, identified by its registration number, with attributes such as type, make, model, colour and owner name. When a report mentions a vehicle that does not yet exist, the system creates a vehicle record automatically. Officers can edit vehicle details and view the full violation history of a vehicle, which helps to identify repeat offenders.

4. Location management

Locations represent junctions or road segments. Storing them as separate records avoids spelling variations such as "MG Road Junction" and "M.G. Road Jn." and allows reliable aggregation. Administrators and officers can add, update and deactivate locations. The report form lets users pick an existing location or enter a new one.

5. Evidence handling

Image evidence is uploaded as multipart form data. The server accepts only JPEG and PNG files up to 5 MB, renames each file with a random identifier to prevent collisions and path manipulation, stores it in the uploads directory and records its path and metadata in the violation document. Images are served through a controlled route and are shown as thumbnails in lists and in full size on the detail page.

6. Review and challan workflow

Officers see a queue of Submitted reports, ordered by date. They open a report, inspect the evidence, and either approve it and set the fine amount (defaulted from the fine schedule) or reject it with a mandatory reason. Approved reports become challans. Closing a report indicates that the fine has been settled. Every transition is appended to a status history array.

7. Search, filter and pagination

Lists can be filtered by status, violation type, location, vehicle number and date range, and sorted by date or fine amount. Server-side pagination ensures that response size and rendering time stay small even when the number of records grows.

8. Dashboard and analytics

The dashboard shows totals for each status, a bar chart of violations by type, a line chart of reports per month, and a ranking of the locations with the most violations. Citizens see a personal summary, while officers and administrators see system-wide figures.

9. Notifications

When the status of a report changes, an in-app notification is created for the reporting citizen. The design allows email or SMS gateways to be added later.

3.3 Functional Requirements

ID	Requirement	Priority
FR-01	The system shall allow a new user to register with a unique email address.	High
FR-02	The system shall authenticate users and issue a JWT on successful login.	High
FR-03	The system shall restrict each API endpoint according to the role of the caller.	High
FR-04	The system shall allow a citizen to submit a violation report with vehicle, type, location, date/time and image.	High
FR-05	The system shall validate the registration number format and reject future dates.	High
FR-06	The system shall accept only JPEG/PNG images of at most 5 MB.	High
FR-07	The system shall generate a unique reference number for every report.	Medium
FR-08	The system shall let a citizen view, edit and delete own reports while they are in Submitted state.	High
FR-09	The system shall let an officer view all reports and filter them.	High
FR-10	The system shall let an officer approve a report with a fine or reject it with a reason.	High
FR-11	The system shall record every status change with actor and time stamp.	Medium
FR-12	The system shall create, read, update and delete vehicle records.	High
FR-13	The system shall create, read, update and deactivate location records.	Medium
FR-14	The system shall let an administrator manage violation types and default fines.	Medium
FR-15	The system shall let an administrator manage user accounts and roles.	Medium
FR-16	The system shall warn about probable duplicate reports.	Medium
FR-17	The system shall paginate list results.	Medium
FR-18	The system shall display analytics through charts and summary cards.	Medium
FR-19	The system shall notify a citizen when the status of a report changes.	Low

Table 3.2: Functional requirements

4. Non-Functional Requirements

- **Security:** passwords are hashed, tokens are signed, inputs are validated and sanitised, and uploads are restricted.
- **Performance:** typical API responses should return within 300 ms on a development machine for lists of up to 10,000 records.
- **Usability:** forms provide inline validation and clear messages; the interface works on screens from 360 px wide.
- **Reliability:** the server handles errors gracefully and returns structured error messages instead of crashing.
- **Scalability:** the stateless API allows horizontal scaling; indexes support common queries.
- **Maintainability:** code is organised in routes, controllers, models and middleware, with consistent naming.
- **Portability:** the application runs on any platform that supports Node.js and MongoDB.

5. Use Case Description

Use case	Actor	Main flow
Register and log in	Citizen, Officer	Enter details; server validates; password hashed; token returned.
Report violation	Citizen	Open form; fill fields; attach image; submit; receive reference number.
Track report	Citizen	Open My Reports; view status badge and history.
Review report	Officer	Open pending queue; inspect evidence; approve with fine or reject with reason.
Manage vehicles	Officer	Search by number; edit details; view violation history.
Configure fines	Administrator	Open violation types; edit default fine; save.
View analytics	Officer, Administrator	Open dashboard; select period; read charts.

Table 3.3: Principal use cases

3.6 Report Life Cycle

The report life cycle is a finite state machine. A new report starts in the Submitted state. An officer moving it into review changes the state to Under Review. From there the officer can Approve or Reject it. An approved report is closed when the fine is paid. A rejected report is closed automatically after a holding period, during which the citizen can view the reason. Transitions that are not listed in Table 3.4 are refused by the server.

From	To	Performed by
Submitted	Under Review	Officer
Submitted	Rejected	Officer
Under Review	Approved	Officer
Under Review	Rejected	Officer
Approved	Closed	Officer / Administrator
Rejected	Closed	Officer / Administrator

Table 3.4: Allowed status transitions

CHAPTER 4: ALGORITHMS AND METHODOLOGY

4.1 Development Methodology

The project follows an iterative, incremental model derived from Agile practice. The work was divided into sprints of about two weeks each, and each sprint ended with a working increment that was demonstrated and reviewed. This model was selected because the requirements became clearer as prototypes were shown to users, and because it allows the most important features to be delivered first.

Sprint	Focus	Deliverable
1	Requirement analysis, database design, project set-up	SRS, ER model, repository, base server
2	Authentication and role-based access	Register/login APIs, JWT middleware, login pages
3	Violation CRUD and image upload	Report form, list, detail page, upload handling
4	Vehicle and location management, review workflow	Officer queue, approve/reject, status history
5	Dashboard, filters, duplicate detection	Charts, pagination, duplicate warning
6	Testing, optimisation, documentation	Test suite, performance report, final report

Table 4.1: Sprint plan

4.2 System Architecture

The system follows a three-tier architecture. The presentation tier is the React single-page application running in the browser. The application tier is the Express.js server running on Node.js, which hosts the REST API, authentication, validation and business logic. The data tier is MongoDB, which stores documents, accessed through the Mongoose object data modelling library. The tiers communicate over HTTP using JSON, except for image upload, which uses multipart form data.

Layer	Component	Responsibility
Presentation	React components, React Router, Context API, Axios	Rendering, form handling, client-side validation, routing, token storage
Application	Express routes, controllers, middleware (auth, role, upload, error)	Request handling, authorisation, validation, business rules, file handling
Data access	Mongoose schemas and models	Schema enforcement, queries, indexes, hooks
Storage	MongoDB database, file system	Persistent records and image files

Table 4.2: Layers of the architecture

The request flow is as follows. The browser sends a request with the JWT in the Authorization header. The Express application passes it through the CORS, JSON-parsing and logging middleware, then through the authentication middleware, which verifies the token and attaches the user to the request. A role middleware checks permission. The controller executes the business logic using Mongoose models and returns a JSON response, which React uses to update the state and redraw the interface.

4.3 Database Design

MongoDB stores data in collections of documents. The design balances embedding, which reads related data in one operation, against referencing, which avoids duplication. Evidence and status history are embedded in the violation document, because they are always read with it and have no independent life. Vehicles,

locations, violation types and users are separate collections referenced by identifier, because they are shared between many violations and are managed independently.

Collection	Fields
users	_id, name, email (unique), phone, passwordHash, role (citizen officer admin), isActive, createdAt
vehicles	_id, regNumber (unique), type, make, model, color, ownerName, createdAt
locations	_id, name, area, city, coordinates {lat, lng}, isActive
violationtypes	_id, name (unique), code, description, defaultFine, isActive
violations	_id, refNo (unique), reporter -> users, vehicle -> vehicles, type -> violationtypes, location -> locations, occurredAt, description, evidence {path, mimeType, size}, status, fineAmount, rejectionReason, history [{status, by, at, note}], createdAt, updatedAt
notifications	_id, user -> users, message, violation -> violations, read, createdAt

Table 4.3: Collection schemas (summary)

1. Indexes

Indexes are created to support the most common queries: a unique index on vehicles.regNumber and users.email; a compound index on violations (status, occurredAt) for officer queues; an index on violations.reporter for the citizen history; and a compound index on violations (vehicle, type, occurredAt) which supports duplicate detection. Without these indexes, MongoDB would have to scan every document for each query, and response times would rise linearly with data size.

2. Relationships

One user can create many violations (one-to-many). One vehicle can have many violations (one-to-many). One violation type and one location can each be associated with many violations. The violation document stores references to these entities. When a list is requested, Mongoose populate operations fetch the necessary fields of the referenced documents, keeping the stored data normalised while providing a rich response to the client.

4. Algorithms

1. Password hashing and verification

Passwords are never stored in readable form. During registration, the password is hashed with bcrypt using ten salt rounds. bcrypt incorporates a random salt and is intentionally slow, which makes brute-force and rainbow-table attacks impractical. During login, the supplied password is compared with the stored hash using bcrypt compare, which is performed in constant time with respect to the content of the password.

Algorithm 4.1: User registration and login

```
REGISTER(name, email, phone, password):
  1. validate that all fields are present and the email format is valid
  2. if a user with this email exists -> return 409 Conflict
  3. hash <- bcrypt.hash(password, 10)
  4. create user {name, email, phone, passwordHash: hash, role: "citizen"}
  5. return 201 Created

LOGIN(email, password):
  1. user <- find user by email
  2. if user is null or not user.isActive -> return 401 Unauthorized
  3. if not bcrypt.compare(password, user.passwordHash) -> return 401
  4. token <- jwt.sign({id: user._id, role: user.role}, SECRET, expiresIn: "1d")
  5. return 200 {token, user}
```

4.4.2 JWT authentication and role-based access control

A JSON Web Token has three parts: a header, a payload and a signature. The signature is computed with a secret known only to the server, so any modification of the payload invalidates the token. This allows the server to authenticate requests without keeping session state. The role in the payload is used by the authorisation middleware. An endpoint declares the roles that may call it, and the middleware compares them with the role of the authenticated user.

Algorithm 4.2: Authentication and authorisation middleware

```
AUTHENTICATE(request):
  1. header <- request.headers.authorization
  2. if header is missing or does not start with "Bearer " -> 401
  3. token <- header after "Bearer "
  4. try payload <- jwt.verify(token, SECRET) catch -> 401
  5. user <- find user by payload.id
  6. if user is null or not user.isActive -> 401
  7. request.user <- user ; continue

AUTHORIZE(allowedRoles):
  1. if request.user.role is not in allowedRoles -> 403 Forbidden
  2. continue
```

4.4.3 Report creation and validation

Creation of a report is a multi-step procedure which ensures that data is valid and that referenced entities exist. The vehicle number is normalised by removing spaces and hyphens and converting to upper case so that "ka 01 ab 1234" and "KA-01-AB-1234" refer to the same vehicle. If the vehicle is not found it is created. The reference number is generated from the year and an incrementing counter.

Algorithm 4.3: Create violation report

```
CREATE_REPORT(user, body, file):
  1. validate body.regNumber against /^[A-Z]{2}[0-9]{1,2}[A-Z]{0,3}[0-9]{4}$/
  2. if body.occurredAt > now() -> 400 "Date cannot be in the future"
  3. if file is missing -> 400 "Evidence image required"
  4. if file.mimetype not in {jpeg, png} or file.size > 5 MB -> 400
  5. reg <- normalise(body.regNumber)
  6. vehicle <- find vehicle by reg ; if null, create it
  7. duplicate <- FIND_DUPLICATE(vehicle, body.type, body.location, body.occurredAt)
  8. if duplicate exists and body.confirm != true -> 409 {duplicate}
  9. refNo <- "TVR-" + year + "-" + zeroPad(nextSequence(), 6)
  10. save violation {refNo, reporter: user, vehicle, type, location,
      occurredAt, description, evidence, status: "Submitted",
      history: [{status: "Submitted", by: user, at: now()}]}
  11. return 201 {violation}
```

4.4.4 Duplicate detection

The same violation may be reported by more than one witness. To reduce unnecessary verification, the system checks whether a report of the same type exists for the same vehicle at the same location within a time window. The window is thirty minutes by default. If a probable duplicate exists, the user is warned and may either cancel or confirm that it is a separate incident. The time complexity is logarithmic in the number of violations because the query uses the compound index on vehicle, type and occurredAt.

Algorithm 4.4: Duplicate detection

```
FIND_DUPLICATE(vehicle, type, location, t):  
  window <- 30 minutes  
  return first violation v where  
    v.vehicle = vehicle  
    and v.type = type  
    and v.location = location  
    and |v.occurredAt - t| <= window  
    and v.status != "Rejected"
```

4.4.5 Status transition validation

The state machine of Section 3.6 is implemented as a lookup table which maps each state to the set of states that may follow. Before any update the server checks that the requested transition is present in the table and that the caller has the appropriate role. This prevents illegal changes such as approving a rejected report directly.

Algorithm 4.5: Status change

```
ALLOWED <- {  
  "Submitted": ["Under Review", "Rejected"],  
  "Under Review": ["Approved", "Rejected"],  
  "Approved": ["Closed"],  
  "Rejected": ["Closed"],  
  "Closed": []  
}  
  
CHANGE_STATUS(report, newStatus, actor, note, fine):  
  1. if newStatus not in ALLOWED[report.status] -> 400 "Invalid transition"  
  2. if newStatus = "Rejected" and note is empty -> 400 "Reason required"  
  3. if newStatus = "Approved":  
    report.fineAmount <- fine if provided else report.type.defaultFine  
  4. report.status <- newStatus  
  5. append {status: newStatus, by: actor, at: now(), note} to report.history  
  6. save report ; create notification for report.reporter
```

4.4.6 Filtering, sorting and pagination

List endpoints build a query object from the request parameters. Only parameters that are supplied are added to the filter. Pagination uses skip and limit. For page p with page size s , the number of documents to skip is $(p - 1) \times s$. The total count is obtained separately so that the client can render page controls. For very large collections, range-based (keyset) pagination on the indexed date field can replace skip-based pagination to avoid the cost of skipping many documents.

Algorithm 4.6: Filtered and paginated listing

```
LIST_REPORTS(params, user):  
  filter <- {}  
  if user.role = "citizen" : filter.reporter <- user._id  
  if params.status : filter.status <- params.status  
  if params.type : filter.type <- params.type  
  if params.location : filter.location <- params.location  
  if params.vehicle : filter.vehicle <- vehicle id of params.vehicle  
  if params.from or params.to:  
    filter.occurredAt <- {gte: params.from, lte: params.to}  
  page <- max(1, params.page) ; size <- min(50, params.limit or 10)  
  sort <- params.sort or "-occurredAt"  
  items <- query(filter).sort(sort).skip((page-1)*size).limit(size).populate(...)
```

```
total <- count(filter)
return {items, page, pages: ceil(total/size), total}
```

4.4.7 Aggregation for analytics

Statistics are computed with the MongoDB aggregation pipeline. A pipeline passes documents through stages such as match, group, sort and project. To count violations by type, the pipeline groups documents by the type reference and counts them, and a lookup stage joins the type name. Aggregation is performed by the database engine, which avoids transferring all documents to the application server.

4.4.8 Image validation and storage

The upload middleware inspects the declared MIME type and the file extension, and rejects files that are not JPEG or PNG. The size limit is enforced while the stream is received, so that oversized files are cancelled early. Files are saved with a random 16-byte hexadecimal name, which prevents guessing and overwriting. The database stores only the relative path and metadata, not the binary content, which keeps documents small.

4.5 Security Methodology

- **Transport:** deployment should use HTTPS so that tokens and personal data are encrypted in transit.
- **Input validation:** every endpoint validates body, query and parameters using schema validators; Mongoose enforces types and required fields.
- **Injection prevention:** MongoDB operators in user input are stripped to prevent NoSQL injection, and output rendered by React is escaped to prevent cross-site scripting.
- **Rate limiting:** login and registration endpoints are limited per IP address to slow brute-force attempts.
- **Headers:** the Helmet middleware sets secure HTTP headers; CORS is restricted to the known frontend origin.
- **Secrets:** the JWT secret and database URI are held in environment variables and never committed to the repository.
- **Least privilege:** citizens can only access their own reports; the server checks ownership in addition to role.

4.6 Frontend Methodology

The React application is organised into pages, reusable components, context providers and service modules. Authentication state is held in a context provider so that any component can read the current user and role. A protected-route component redirects unauthenticated users to the login page and shows an access-denied message when the role is not permitted. API calls are centralised in an Axios instance that adds the token to every request and handles expired tokens by logging the user out. Form state is held in controlled components with validation messages displayed next to each field. Lists use server-side pagination and debounce text filters to avoid sending a request for every keystroke.

CHAPTER 5: TECHNOLOGIES AND IMPLEMENTATION

5.1 Technology Stack

Technology	Role in the project
MongoDB	Document database storing users, vehicles, locations, violation types and violations
Express.js	Web framework for routing, middleware and REST API
React.js	Component-based library for the single-page user interface
Node.js	JavaScript runtime that executes the server
Mongoose	Object data modelling, validation and query building for MongoDB
jsonwebtoken	Creation and verification of JWTs
bcryptjs	Password hashing
Multer	Multipart form data and file upload handling
express-validator	Request validation and sanitisation
Helmet, CORS, express-rate-limit	HTTP security headers, cross-origin policy, request throttling
React Router	Client-side routing and protected routes
Axios	HTTP client with interceptors
Chart.js	Charts on the dashboard
Jest, Supertest, Postman	Unit tests, API tests and manual API verification
VS Code, Git, MongoDB Compass	Development environment, version control and database inspection

Table 5.1: Technologies used

Node.js uses an event-driven, non-blocking I/O model, which allows a single process to serve many concurrent requests efficiently, and it is a good fit for an API server whose work consists mostly of database and file operations. MongoDB stores data in BSON documents, which map directly to JavaScript objects, so no object-relational translation is required. React updates only the parts of the page that change by using a virtual DOM, which keeps the interface responsive.

5.2 Project Structure

Listing 5.1: Folder layout

```
traffic-violation-system/
|-- server/
|   |-- config/db.js
|   |-- controllers/
|   |   |-- authController.js
|   |   |-- violationController.js
|   |   |-- vehicleController.js
|   |   |-- locationController.js
|   |   |-- statsController.js
|   |-- middleware/
|   |   |-- auth.js           (authenticate, authorize)
|   |   |-- upload.js        (multer configuration)
|   |   |-- errorHandler.js
|   |-- models/
|   |   |-- User.js  Vehicle.js  Location.js  ViolationType.js  Violation.js
|   |-- routes/
|   |   |-- authRoutes.js  violationRoutes.js  vehicleRoutes.js ...
|   |-- uploads/
```

```

|   |-- .env
|   `-- server.js
|-- client/
|   |-- src/
|       |-- api/axios.js
|       |-- context/AuthContext.jsx
|       |-- components/ (Navbar, ProtectedRoute, StatusBadge, Pagination)
|       |-- pages/ (Login, Register, ReportForm, MyReports, ReviewQueue,
|                   ReportDetail, Vehicles, Locations, Dashboard, Admin)
|       |-- App.jsx
|       `-- main.jsx
|-- package.json

```

3. Backend Implementation

1. Server entry point

Listing 5.2: server.js

```

require('dotenv').config();
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
const path = require('path');
const connectDB = require('./config/db');
const errorHandler = require('./middleware/errorHandler');

const app = express();
connectDB();

app.use(helmet());
app.use(cors({ origin: process.env.CLIENT_URL }));
app.use(express.json({ limit: '1mb' }));
app.use('/api/auth', rateLimit({ windowMs: 15 * 60 * 1000, max: 50 }));

app.use('/uploads', express.static(path.join(__dirname, 'uploads')));
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/violations', require('./routes/violationRoutes'));
app.use('/api/vehicles', require('./routes/vehicleRoutes'));
app.use('/api/locations', require('./routes/locationRoutes'));
app.use('/api/types', require('./routes/typeRoutes'));
app.use('/api/stats', require('./routes/statsRoutes'));

app.use(errorHandler);
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log('Server running on port ' + PORT));

```

Listing 5.3: config/db.js

```

const mongoose = require('mongoose');
module.exports = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB connected');
  } catch (err) {
    console.error('Database connection failed:', err.message);
    process.exit(1);
  }
}

```



```
};
```

5.3.2 Data models

Listing 5.4: models/User.js

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name:      { type: String, required: true, trim: true },
  email:     { type: String, required: true, unique: true, lowercase: true },
  phone:     { type: String, match: /^[0-9]{10}$/ },
  password:  { type: String, required: true, minlength: 8, select: false },
  role:      { type: String, enum: ['citizen', 'officer', 'admin'], default: 'citizen' },
  isActive:  { type: Boolean, default: true }
}, { timestamps: true });

userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});

userSchema.methods.matches = function (plain) {
  return bcrypt.compare(plain, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

Listing 5.5: models/Violation.js

```
const mongoose = require('mongoose');

const historySchema = new mongoose.Schema({
  status: String,
  by:      { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  at:      { type: Date, default: Date.now },
  note:    String
}, { _id: false });

const violationSchema = new mongoose.Schema({
  refNo:      { type: String, unique: true },
  reporter:   { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  vehicle:    { type: mongoose.Schema.Types.ObjectId, ref: 'Vehicle', required: true },
  type:       { type: mongoose.Schema.Types.ObjectId, ref: 'ViolationType', required: true },
  location:   { type: mongoose.Schema.Types.ObjectId, ref: 'Location', required: true },
  occurredAt: { type: Date, required: true },
  description: { type: String, maxlength: 500 },
  evidence:   { path: String, mimeType: String, size: Number },
  status:     { type: String,
    enum: ['Submitted', 'Under Review', 'Approved', 'Rejected', 'Closed'],
    default: 'Submitted' },
  fineAmount: { type: Number, min: 0 },
  rejectionReason: String,
  history:    [historySchema]
}, { timestamps: true });

violationSchema.index({ status: 1, occurredAt: -1 });
violationSchema.index({ reporter: 1, createdAt: -1 });
violationSchema.index({ vehicle: 1, type: 1, occurredAt: 1 });
module.exports = mongoose.model('Violation', violationSchema);
```

5.3.3 Authentication and authorisation middleware

Listing 5.7: middleware/auth.js

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

exports.authenticate = async (req, res, next) => {
  try {
    const header = req.headers.authorization || '';
    if (!header.startsWith('Bearer ')) {
      return res.status(401).json({ message: 'Authentication required' });
    }
    const payload = jwt.verify(header.split(' ')[1], process.env.JWT_SECRET);
    const user = await User.findById(payload.id);
    if (!user || !user.isActive) {
      return res.status(401).json({ message: 'Invalid account' });
    }
    req.user = user;
    next();
  } catch (err) {
    res.status(401).json({ message: 'Invalid or expired token' });
  }
};

exports.authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ message: 'Access denied' });
  }
  next();
};
```

Listing 5.8: controllers/authController.js

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const sign = (user) => jwt.sign({ id: user._id, role: user.role },
  process.env.JWT_SECRET, { expiresIn: '1d' });

exports.register = async (req, res, next) => {
  try {
    const { name, email, phone, password } = req.body;
    if (await User.findOne({ email })) {
      return res.status(409).json({ message: 'Email already registered' });
    }
    const user = await User.create({ name, email, phone, password });
    res.status(201).json({ token: sign(user),
      user: { id: user._id, name, email, role: user.role } });
  } catch (err) { next(err); }
};

exports.login = async (req, res, next) => {
  try {
    const user = await User.findOne({ email: req.body.email }).select('+password');
    if (!user || !user.isActive || !(await user.matches(req.body.password))) {
      return res.status(401).json({ message: 'Invalid email or password' });
    }
  }
};
```

```

    user: { id: user._id, name: user.name, email: user.email, role: user.role } });
  } catch (err) { next(err); }
};

```

5.3.4 File upload middleware

Listing 5.9: middleware/upload.js

```

const multer = require('multer');
const crypto = require('crypto');
const path = require('path');

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, 'uploads/'),
  filename: (req, file, cb) =>
    cb(null, crypto.randomBytes(16).toString('hex') +
    path.extname(file.originalname).toLowerCase())
});

const fileFilter = (req, file, cb) => {
  const ok = ['image/jpeg', 'image/png'].includes(file.mimetype);
  cb(ok ? null : new Error('Only JPEG and PNG images are allowed'), ok);
};

module.exports = multer({ storage, fileFilter, limits: { fileSize: 5 * 1024 * 1024 } });

```

5.3.5 Violation controller

Listing 5.10: controllers/violationController.js (create and list)

```

const Violation = require('../models/Violation');
const Vehicle = require('../models/Vehicle');

const normalise = (s) => s.replace(/\s-/g, '').toUpperCase();

exports.create = async (req, res, next) => {
  try {
    const { regNumber, type, location, occurredAt, description, confirm } = req.body;
    if (!req.file) return res.status(400).json({ message: 'Evidence image required' });
    if (new Date(occurredAt) > new Date()) {
      return res.status(400).json({ message: 'Date cannot be in the future' });
    }
    const reg = normalise(regNumber);
    const vehicle = await Vehicle.findOneAndUpdate(
      { regNumber: reg }, { $setOnInsert: { regNumber: reg } },
      { upsert: true, new: true });

    const t = new Date(occurredAt), win = 30 * 60 * 1000;
    const dup = await Violation.findOne({
      vehicle: vehicle._id, type, location,
      occurredAt: { $gte: new Date(t - win), $lte: new Date(t + win) },
      status: { $ne: 'Rejected' } });
    if (dup && confirm !== 'true') {
      return res.status(409).json({ message: 'Possible duplicate', duplicate: dup.refNo });
    }

    const count = await Violation.countDocuments();
    const refNo = 'TVR-' + new Date().getFullYear() + '-' + String(count + 1).padStart(6, '0');
    const v = await Violation.create({

```

```

        occurredAt, description,
        evidence: { path: req.file.filename, mimeType: req.file.mimetype, size: req.file.size },
        history: [{ status: 'Submitted', by: req.user._id }] });
    res.status(201).json(v);
  } catch (err) { next(err); }
};

exports.list = async (req, res, next) => {
  try {
    const { status, type, location, from, to, page = 1, limit = 10, sort = '-occurredAt' } =
req.query;
    const filter = {};
    if (req.user.role === 'citizen') filter.reporter = req.user._id;
    if (status) filter.status = status;
    if (type) filter.type = type;
    if (location) filter.location = location;
    if (from || to) filter.occurredAt = { ...(from && { $gte: from }), ...(to && { $lte: to }) };

    const size = Math.min(50, Number(limit)), p = Math.max(1, Number(page));
    const [items, total] = await Promise.all([
      Violation.find(filter).sort(sort).skip((p - 1) * size).limit(size)
        .populate('vehicle type location', 'regNumber name area defaultFine'),
      Violation.countDocuments(filter)
    ]);
    res.json({ items, page: p, pages: Math.ceil(total / size), total });
  } catch (err) { next(err); }
};

```

Listing 5.11: controllers/violationController.js (update, status, delete)

```

const ALLOWED = {
  'Submitted': ['Under Review', 'Rejected'],
  'Under Review': ['Approved', 'Rejected'],
  'Approved': ['Closed'], 'Rejected': ['Closed'], 'Closed': []
};

exports.update = async (req, res, next) => { // citizen edits own pending report
  try {
    const v = await Violation.findById(req.params.id);
    if (!v) return res.status(404).json({ message: 'Report not found' });
    if (String(v.reporter) !== String(req.user._id) || v.status !== 'Submitted') {
      return res.status(403).json({ message: 'Report cannot be edited' });
    }
    ['description', 'location', 'type', 'occurredAt'].forEach(k => {
      if (req.body[k] !== undefined) v[k] = req.body[k];
    });
    res.json(await v.save());
  } catch (err) { next(err); }
};

exports.changeStatus = async (req, res, next) => { // officer / admin
  try {
    const { status, note, fine } = req.body;
    const v = await Violation.findById(req.params.id).populate('type');
    if (!v) return res.status(404).json({ message: 'Report not found' });
    if (!ALLOWED[v.status].includes(status)) {
      return res.status(400).json({ message: 'Invalid transition' });
    }
    if (status === 'Rejected') {
      if (!note) return res.status(400).json({ message: 'Reason required' });
      v.rejectionReason = note;
    }
  }
};

```

```

    }
    if (status === 'Approved') v.fineAmount = fine ?? v.type.defaultFine;
    v.status = status;
    v.history.push({ status, by: req.user._id, note });
    res.json(await v.save());
  } catch (err) { next(err); }
};

exports.remove = async (req, res, next) => {
  try {
    const v = await Violation.findById(req.params.id);
    if (!v) return res.status(404).json({ message: 'Report not found' });
    const owner = String(v.reporter) === String(req.user._id);
    if (!(req.user.role === 'admin' || (owner && v.status === 'Submitted'))) {
      return res.status(403).json({ message: 'Not allowed' });
    }
    await v.deleteOne();
    res.json({ message: 'Deleted' });
  } catch (err) { next(err); }
};

```

5.3.6 REST API summary

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Register a new citizen
POST	/api/auth/login	Public	Log in and receive a token
GET	/api/auth/me	Authenticated	Get current user profile
PUT	/api/auth/me	Authenticated	Update own profile
POST	/api/violations	Citizen	Submit a report with image
GET	/api/violations	Authenticated	List reports with filters and pagination
GET	/api/violations/:id	Authenticated	Get one report
PUT	/api/violations/:id	Citizen (owner)	Edit a pending report
PATCH	/api/violations/:id/status	Officer, Admin	Approve, reject or close
DELETE	/api/violations/:id	Owner / Admin	Delete a report
GET	/api/vehicles	Officer, Admin	Search vehicles
POST	/api/vehicles	Officer, Admin	Create a vehicle
PUT	/api/vehicles/:id	Officer, Admin	Update a vehicle
DELETE	/api/vehicles/:id	Admin	Delete a vehicle
GET	/api/locations	Authenticated	List locations
POST	/api/locations	Officer, Admin	Create a location
PUT	/api/locations/:id	Officer, Admin	Update a location
DELETE	/api/locations/:id	Admin	Deactivate a location
GET/POST/ PUT	/api/types	Admin (write)	Manage violation types and fines
GET	/api/stats/type	Officer, Admin	Counts by violation type
GET	/api/stats/month	Officer, Admin	Counts by month
GET/PUT	/api/users	Admin	Manage users and roles

Table 5.2: REST API endpoints

4. Frontend Implementation

1. Axios instance and authentication context

Listing 5.14: src/api/axios.js

```
import axios from 'axios';

const api = axios.create({ baseURL: import.meta.env.VITE_API_URL });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = 'Bearer ' + token;
  return config;
});

api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(err);
  }
);

export default api;
```

Listing 5.15: src/context/AuthContext.jsx and ProtectedRoute

```
import { createContext, useContext, useState } from 'react';
import { Navigate } from 'react-router-dom';
import api from '../api/axios';

const AuthContext = createContext();
export const useAuth = () => useContext(AuthContext);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(JSON.parse(localStorage.getItem('user')) || 'null');

  const login = async (email, password) => {
    const { data } = await api.post('/auth/login', { email, password });
    localStorage.setItem('token', data.token);
    localStorage.setItem('user', JSON.stringify(data.user));
    setUser(data.user);
  };

  const logout = () => { localStorage.clear(); setUser(null); };

  return <AuthContext.Provider value={{ user, login, logout }}>{children}</AuthContext.Provider>;
}

export function ProtectedRoute({ roles, children }) {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" replace />;
  if (roles && !roles.includes(user.role)) return <p>Access denied</p>;
  return children;
}
```

5.4.2 Report form

Listing 5.16: src/pages/ReportForm.jsx

```
import { useEffect, useState } from 'react';
import api from '../api/axios';

export default function ReportForm() {
  const [types, setTypes] = useState([]);
  const [locations, setLocations] = useState([]);
  const [form, setForm] = useState({ regNumber: '', type: '', location: '',
                                     occurredAt: '', description: '' });
  const [file, setFile] = useState(null);
  const [msg, setMsg] = useState('');

  useEffect(() => {
    api.get('/types').then(r => setTypes(r.data));
    api.get('/locations').then(r => setLocations(r.data));
  }, []);

  const change = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  const submit = async (confirm = false) => {
    if (!/^[A-Z]{2}[0-9]{1,2}[A-Z]{0,3}[0-9]{4}$/.test(form.regNumber.replace(/\s-/g,
    '').toUpperCase())) {
      return setMsg('Enter a valid registration number');
    }
    if (!file) return setMsg('Please attach an image');
    const fd = new FormData();
    Object.entries(form).forEach(([k, v]) => fd.append(k, v));
    fd.append('evidence', file);
    if (confirm) fd.append('confirm', 'true');
    try {
      const { data } = await api.post('/violations', fd);
      setMsg('Report submitted. Reference: ' + data.refNo);
    } catch (err) {
      if (err.response?.status === 409 &&
          window.confirm('A similar report (' + err.response.data.duplicate +
                          ') exists. Submit anyway?')) return submit(true);
      setMsg(err.response?.data?.message || 'Submission failed');
    }
  };

  return (
    <div className="card">
      <h2>Report a Traffic Violation</h2>
      <input name="regNumber" placeholder="Vehicle number" onChange={change} />
      <select name="type" onChange={change}>
        <option value="">Violation type</option>
        {types.map(t => <option key={t._id} value={t._id}>{t.name}</option>)}
      </select>
      <select name="location" onChange={change}>
        <option value="">Location</option>
        {locations.map(l => <option key={l._id} value={l._id}>{l.name}, {l.area}</option>)}
      </select>
      <input type="datetime-local" name="occurredAt" max={new Date().toISOString().slice(0,16)}
        onChange={change} />
      <textarea name="description" maxLength="500" onChange={change} />
      <input type="file" accept="image/jpeg,image/png" onChange={e =>
setFile(e.target.files[0])} />
      <button onClick={() => submit()}>Submit</button>
    </div>
  );
}
```

```

    <p>{msg}</p>
  </div>
);
}

```

5.4.3 Officer review queue and routing

The review queue page loads reports with the status filter set to Submitted, shows evidence thumbnails in a table, and calls the status endpoint when the officer chooses Review, Approve or Reject; rejection prompts for a mandatory reason. Routing is declared in App.jsx with React Router, and each route is wrapped in the ProtectedRoute component of Listing 5.15 with the list of roles allowed to open it: citizens see the report form and My Reports, officers see the review queue, vehicles and dashboard, and administrators additionally see user management.

5.5 User Interface Description

The interface is organised around the task of each role. All screens share a top navigation bar that shows the application name, links that depend on the role, and the name of the logged-in user with a logout option. Status badges use colour coding with text labels: blue for Submitted, amber for Under Review, green for Approved, red for Rejected and grey for Closed.

Screen	Users	Contents
Login / Register	All	Email and password fields with validation messages; link between the two screens.
Report Violation	Citizen	Vehicle number, type, location drop-downs, date-time picker, description, image chooser with preview.
My Reports	Citizen	Paginated cards or table with reference number, status badge, date and thumbnail; edit and delete for pending items.
Report Detail	All	Full evidence image, all fields, status timeline, fine amount and rejection reason.
Review Queue	Officer	Filterable table of reports with quick approve, reject and review buttons.
Vehicles	Officer	Search box, vehicle table, edit form and violation history per vehicle.
Locations and Types	Officer, Admin	CRUD forms for locations, violation types and default fines.
Dashboard	Officer, Admin	Status cards, bar chart by type, line chart by month, top locations.
User Management	Admin	List of users with role selector and activate/deactivate switch.

Table 5.3: Principal screens

5.6 Deployment and Configuration

The server and client are configured through environment variables. The server needs MONGO_URI, JWT_SECRET, CLIENT_URL and PORT. The client needs VITE_API_URL. For development, MongoDB runs locally or on a free cloud cluster, the server is started with nodemon and the client with the Vite development server. For production, the React application is compiled into static files and served by a static host or by the Express server itself, the Node process is managed by a process manager such as PM2, and a reverse proxy terminates HTTPS. Uploaded images should be moved to object storage when the system is scaled to more than one server.

CHAPTER 6: RESULTS, TESTING AND PERFORMANCE

6.1 Testing Strategy

Testing was planned at four levels. Unit tests verified individual functions such as registration number normalisation, the status transition table and the fine calculation. API tests exercised each endpoint through Supertest against a test database, checking status codes, response bodies and authorisation rules. Functional (system) tests followed the use cases from the user interface, and non-functional tests examined security and performance. Postman collections were used throughout development for quick manual checks.

Level	Tool	Scope
Unit	Jest	Helpers, validators, transition table
Integration / API	Jest, Supertest, Postman	All REST endpoints, middleware, database interaction
Functional / UI	Manual test scripts, browser developer tools	End-to-end flows per role
Security	Manual attempts, Postman	Unauthorised access, invalid tokens, malicious files
Performance	Postman runner, autocannon	Response time and throughput under load

Table 6.1: Test levels

6.2 Test Cases and Results

ID	Test case	Expected result	Result
TC-01	Register with valid details	201 Created; token returned	Pass
TC-02	Register with existing email	409 Conflict	Pass
TC-03	Login with correct credentials	200 OK with token	Pass
TC-04	Login with wrong password	401 Unauthorized	Pass
TC-05	Access protected API without token	401 Unauthorized	Pass
TC-06	Access with tampered token	401 Unauthorized	Pass
TC-07	Citizen opens officer endpoint	403 Forbidden	Pass
TC-08	Submit report with all valid data	201 Created; refNo generated	Pass
TC-09	Submit without image	400 Bad Request	Pass
TC-10	Submit with PDF file	Rejected with message	Pass
TC-11	Submit image larger than 5 MB	Rejected (file too large)	Pass
TC-12	Submit with future date	400 Bad Request	Pass
TC-13	Submit with invalid registration number	400 / form error shown	Pass
TC-14	Submit duplicate within 30 minutes	409 with duplicate ref; confirm allows	Pass
TC-15	Citizen lists reports	Only own reports returned	Pass
TC-16	Citizen opens another user's report by id	403 / not returned	Pass
TC-17	Citizen edits own Submitted report	200 OK; fields updated	Pass
TC-18	Citizen edits an Approved report	403 Forbidden	Pass
TC-19	Officer approves report with fine	Status Approved; fine stored; history added	Pass
TC-20	Officer rejects without reason	400 Bad Request	Pass
TC-21	Officer changes Rejected to Approved	400 Invalid transition	Pass

ID	Test case	Expected result	Result
TC-22	Filter by status and date range	Only matching records	Pass
TC-23	Pagination page 2 with limit 10	Records 11 to 20; correct page count	Pass
TC-24	Create, update and delete vehicle	CRUD operations succeed	Pass
TC-25	Admin changes a user role	Role updated; new token reflects role	Pass
TC-26	Dashboard statistics	Counts equal records in database	Pass

Table 6.2: Functional test cases

All twenty-six functional test cases produced the expected outcome in the final build. During development, several defects were found and fixed. For example, the first version of the duplicate check ignored rejected reports, which caused a legitimate resubmission to be blocked; this was corrected by excluding the Rejected status from the query. Another defect allowed an officer to approve a report directly from the Submitted state; this was fixed by introducing the transition table.

6.3 Results

The completed system provides the full workflow described in the objectives. A citizen can register, log in, and submit a report that contains the vehicle number, violation type, location, date and time and an image. The report receives a reference number and appears immediately in the My Reports page with the Submitted status. An officer sees the report in the review queue, opens it to inspect the evidence, and approves it with a fine or rejects it with a reason. The citizen sees the updated status and the history of changes. The administrator can maintain violation types, fines, locations and users, and can consult the dashboard.

Objective	Outcome
Demonstrate MERN development	Achieved: all four technologies are integrated and used according to their strengths.
Implement CRUD with MongoDB	Achieved: CRUD on users, vehicles, locations, types and violations.
Build REST APIs	Achieved: more than twenty endpoints with validation and consistent errors.
Develop React frontend	Achieved: responsive pages for each role with forms, tables and charts.
Authentication and role-based access	Achieved: JWT login, role middleware and ownership checks.
Image evidence	Achieved: validated upload, storage and display of evidence.
Clear display of information	Achieved: status badges, timeline, dashboard summaries and charts.

Table 6.3: Achievement of objectives

6.3.1 Description of output screens

Citizen dashboard: The landing page shows three summary cards (total reports, pending, approved) and a table of recent reports. A prominent button leads to the report form.

Report form: The form is laid out in a single column on mobile and two columns on desktop. Selecting an image displays a preview. Error messages appear directly under the invalid field. After a successful submission a green confirmation shows the reference number.

Review queue: The officer sees thumbnails of the evidence in the table, so that obvious cases can be handled without opening them. The filters above the table reduce the list by status, type, location and date.

Report detail: The detail page shows the full-size image on the left, the report data on the right, and a vertical timeline of status changes below. Officers also see the action buttons and the fine field.

Dashboard: A bar chart displays the number of reports by violation type, a line chart shows monthly trends, and a ranked list shows the locations with the most reports. These figures help to decide where to deploy personnel.

Screenshots of each of these screens may be inserted here from the running application to accompany the descriptions above.

6.4 Performance Evaluation

Performance was evaluated on a development laptop with the server, database and test client on the same machine. A dataset of ten thousand generated violation records was loaded. The measurements in Table 6.4 are indicative values for this environment, and they will differ on other hardware and deployments. Each request type was executed one hundred times and the median response time is reported.

Operation	Median response (ms)	Remarks
Login (bcrypt, 10 rounds)	110	Dominated by hashing cost, which is intentional
List reports, page of 10, no filter	18	Uses (status, occurredAt) index
List reports with status and date filter	22	Index range scan
Get single report with populate	9	Primary key lookup
Create report with 1 MB image	45	Includes file write and duplicate check
Change status	14	Single document update
Statistics by type (aggregation)	35	Grouping on the server side

Table 6.4: Indicative API response times (ten thousand records)

6.4.1 Effect of indexing

To demonstrate the value of indexes, the filtered list query was run with and without the compound index on status and date. Without the index, MongoDB performed a collection scan that examined all documents, and the response time grew in proportion to the data size. With the index, only the matching index range was visited. The explain output confirmed the change from a collection scan to an index scan, with a large reduction in the number of documents examined.

Records	Without index (ms)	With index (ms)
1,000	6	4
10,000	38	22
50,000	190	25

Table 6.5: Indicative effect of indexing on the filtered list

6.4.2 Load behaviour

A load test with fifty concurrent connections requesting the list endpoint for thirty seconds completed without errors, and the response times remained in the same order of magnitude as the single-user case. Image upload was the most resource-intensive operation, since it involves disk writes, but it remained within acceptable limits for the expected usage. Since the API is stateless, additional server instances behind a load balancer would increase capacity, provided that images are moved to shared storage.

6.4.3 Frontend performance

The React application is split into route-level chunks that are loaded on demand, so a citizen does not download the charting library used only by officers. Lists render only the current page of results, and text filters are debounced by 400 milliseconds. Thumbnails are displayed with fixed dimensions to prevent layout shifts, and images use lazy loading.

6.5 Security Testing

Check	Method	Observation
Password storage	Inspect database	Only bcrypt hashes stored
Token tampering	Modify payload role to admin	Rejected: signature invalid
Expired token	Use token past expiry	Rejected with 401; client redirects to login
Horizontal privilege escalation	Request another citizen's report	Denied by ownership check
Vertical privilege escalation	Citizen calls status endpoint	403 Forbidden
Malicious upload	Upload .js and .exe renamed to .png	Rejected by type check; randomised names
NoSQL injection	Send {"\$gt": ""} as email	Operator stripped; login fails
Brute-force login	Repeated attempts	Throttled by rate limiter

Table 6.6: Security checks

6.6 Discussion

The results show that the chosen architecture meets the stated objectives. The separation of routes, controllers and models made the backend easy to test and extend. Embedding the history and evidence in the violation document made reads fast, while referencing shared entities avoided duplicated data. The status transition table proved valuable as a single place where the business rules of the workflow are defined. The main limitations are that evidence authenticity is judged by human officers, that vehicle details are not verified against an official database, and that images are stored on the local disk. These are addressed in the future scope.

CHAPTER 7: CONCLUSION, FUTURE SCOPE AND REFERENCES

1. Conclusion

This project successfully designed and implemented a Traffic Violation Reporting System using the MERN stack. The system lets citizens report violations with structured data and photographic evidence, gives officers an efficient queue in which to verify and act on reports, and provides administrators with tools for configuration and analysis. It demonstrates CRUD operations with MongoDB, a RESTful API built with Node.js and Express.js, a responsive React interface, and secure role-based access using JWT.

The work shows that a full-stack JavaScript application can address a practical civic problem at low cost. The structured schema removes the ambiguity of informal complaints, the life-cycle model gives accountability and transparency, and the duplicate detection and filtering features reduce the effort of verification. Testing confirmed the functional behaviour and showed that the system performs well for the intended scale. Beyond the technical outcome, the project provided experience of the complete software development life cycle, including requirement analysis, data modelling, API design, security, testing and documentation.

It must be emphasised that the system is a reporting and verification aid. It does not replace the authority of traffic enforcement agencies, and the final decision on every penalty rests with officers acting under the law.

2. Limitations

- Evidence is verified manually; the system does not analyse images automatically.
- Vehicle ownership details are entered or inferred rather than fetched from an official registry.
- Payment of fines is simulated by changing the status to Closed.
- Images are stored on the local file system, which limits horizontal scaling.
- Notifications are shown in the application only.

3. Future Scope

- **Automatic number-plate recognition:** use computer vision to extract the registration number from the uploaded image and pre-fill the form, reducing typing errors.
- **Automatic violation detection:** apply machine learning models to detect helmetless riding, triple riding or signal jumping from images or video.
- **Mobile application:** build a React Native client with camera integration, automatic capture of GPS coordinates and time stamp, and push notifications.
- **Map integration:** show violations on an interactive map with heat maps to identify hotspots and to guide patrolling.
- **Official integrations:** connect to the vehicle registration database and to online payment gateways for real challans and settlement.
- **Cloud storage:** store images in object storage with a content delivery network, and generate thumbnails automatically.
- **Video evidence:** allow short video clips with size limits and server-side compression.
- **Notifications:** send email and SMS alerts, and WebSocket-based real-time updates.
- **Reputation and rewards:** give trusted reporters a credibility score, and use it to prioritise their reports; recognise good contributors.

- **Predictive analytics:** forecast violation trends and recommend enforcement schedules using historical data.
- **Multilingual interface:** support regional languages to increase accessibility.
- **Privacy features:** automatic blurring of faces and unrelated number plates in evidence images, and data retention policies.
- **Blockchain-based evidence ledger:** store cryptographic hashes of evidence to prove that files were not altered after submission.

7.4 References

- 1 MongoDB Inc., "MongoDB Manual", <https://www.mongodb.com/docs/manual/>
- 2 Express.js, "Express - Node.js web application framework", <https://expressjs.com/>
- 3 Meta Platforms, "React Documentation", <https://react.dev/>
- 4 OpenJS Foundation, "Node.js Documentation", <https://nodejs.org/en/docs>
- 5 Mongoose, "Mongoose ODM Documentation", <https://mongoosejs.com/docs/>
- 6 M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)", RFC 7519, IETF, 2015.
- 7 N. Provos and D. Mazieres, "A Future-Adaptable Password Scheme", Proceedings of the USENIX Annual Technical Conference, 1999.
- 8 R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures", Doctoral dissertation, University of California, Irvine, 2000.
- 9 OWASP Foundation, "OWASP Top Ten Web Application Security Risks", <https://owasp.org/www-project-top-ten/>
- 10 Multer, "Middleware for handling multipart/form-data", <https://github.com/expressjs/multer>
- 11 Axios, "Axios Documentation", <https://axios-http.com/docs/intro>
- 12 Chart.js, "Chart.js Documentation", <https://www.chartjs.org/docs/>
- 13 Government of India, "The Motor Vehicles Act, 1988 and the Motor Vehicles (Amendment) Act, 2019".
- 14 Ministry of Road Transport and Highways, Government of India, "Road Accidents in India" (annual publication).
- 15 I. Sommerville, "Software Engineering", 10th Edition, Pearson, 2015.
- 16 K. Schwaber and J. Sutherland, "The Scrum Guide", 2020, <https://scrumguide.org/>